

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA0301

COURSE NAME: Reinforcement learning for Environment

COURSE OUTCOME COVERED

***CO3:** Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

Solution:

Problem Statement

Estimate how a decision-making problem can be modeled as a **Markov Decision Process (MDP)** by defining the action space, transition probability, reward function, and discount factor. The model helps an agent select the best action to maximize the expected cumulative reward.

Objectives

- Understand MDP components.
- Define states and actions.
- Define transition probabilities.
- Define the reward function.
- Apply a discount factor.

- Select the best action based on expected reward.

Eligibility

- Basic knowledge of Artificial Intelligence and Reinforcement Learning.
- Basic Python programming knowledge.
- Basic understanding of probability.

Challenge Rules

- Define states, actions, transitions, rewards, and discount factor correctly.
- Transition probabilities must be between 0 and 1.
- Probabilities for each state-action pair must sum to 1.
- Discount factor must be between 0 and 1.
- The solution should produce a valid decision/policy.

Programming Languages

Python 3.x

Environment Setup

- Python 3.x
- Jupyter Notebook / Google Colab / GitHub
- No external libraries are required.

Input Format

The MDP consists of:

- States
- Actions
- Transition probabilities
- Rewards
- Discount factor

Example:

States: A, B, Goal

Actions: Left, Right

Discount Factor: 0.9

Code

```
# Simple Markov Decision Process
states = ["A", "B", "Goal"]
actions = ["Left", "Right"]
# Reward for each state-action
reward = {
    "A": {"Left": 1, "Right": 5},
    "B": {"Left": 2, "Right": 10},
    "Goal": {"Left": 0, "Right": 0}
}
# Discount factor
gamma = 0.9
# Select best action
for state in states:
    if state == "Goal":
        print(state, "-> Stop")
    else:
        best_action = max(actions, key=lambda a: reward[state][a])
        print(state, "->", best_action,
              "Reward =", reward[state][best_action])
print("Discount Factor =", gamma)
```

Output Format

<pre>1 # Simple Markov Decision Process 2 states = ["A", "B", "Goal"] 3 actions = ["Left", "Right"] 4 # Reward for each state-action 5 reward = { 6 "A": {"Left": 1, "Right": 5}, 7 "B": {"Left": 2, "Right": 10}, 8 "Goal": {"Left": 0, "Right": 0} 9 } 10 # Discount factor 11 gamma = 0.9 12 # Select best action 13 for state in states: 14 if state == "Goal": 15 print(state, "-> Stop") 16 else: 17 best_action = max(actions, key=lambda a: 18 reward[state][a]) 19 print(state, "->", best_action, 20 "Reward =", reward[state][best_action]) 21 print("Discount Factor =", gamma)</pre>	<pre>A -> Right Reward = 5 B -> Right Reward = 10 Goal -> Stop Discount Factor = 0.9 === Code Execution Successful ===</pre>
---	--

Constraints

- $0 \leq P(s'|s,a) \leq 1$
- Transition probabilities for each state-action pair must sum to 1.
- $0 \leq \gamma \leq 1$
- Rewards must be finite values.
- Actions must belong to the defined action space.

2. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency

Solution:

Problem Statement

Justify the **exploration–exploitation trade-off** in Reinforcement Learning (RL) and compare **ϵ -greedy** and **Softmax** action-selection strategies. The aim is to understand how an RL agent balances trying new actions (**exploration**) with selecting the best-known action (**exploitation**) to improve learning efficiency.

Objectives

- Understand exploration and exploitation in RL.
- Explain the exploration–exploitation trade-off.
- Implement the ϵ -greedy strategy.
- Implement the Softmax strategy.
- Compare their action-selection behavior.
- Identify which strategy provides better learning efficiency in different situations.

Eligibility

- Basic knowledge of Reinforcement Learning.
- Basic Python programming knowledge.
- Understanding of rewards and action selection.

Challenge Rules

- The agent must perform both exploration and exploitation.
- ϵ must be between 0 and 1.
- Higher-valued actions should have a higher probability in Softmax.
- The strategies should be compared using the same environment/rewards.
- The program should clearly display the selected actions.

Programming Languages

Python 3.x

Environment Setup

- Python 3.x

- Jupyter Notebook / Google Colab / GitHub
- No external libraries are required.

Input Format

The input contains action values and exploration parameters.

Actions: A, B, C

Q-values: 2, 5, 3

Epsilon: 0.2

Temperature: 1.0

Code

```
import random
import math

actions = ["A", "B", "C"]
q = [2, 5, 3]
epsilon = 0.2
temperature = 1.0
# Epsilon-Greedy
if random.random() < epsilon:
    eg_action = random.choice(actions)
else:
    eg_action = actions[q.index(max(q))]
# Softmax
prob = [math.exp(x / temperature) for x in q]
total = sum(prob)
prob = [x / total for x in prob]
soft_action = random.choices(actions, weights=prob)[0]
print("Q-values:", q)
print("Epsilon-Greedy Action:", eg_action)
print("Softmax Action:", soft_action)
print("Softmax Probabilities:", prob)
```

Output Format

```
main.py  Run  Clear
```

```
1 import random
2 import math
3
4 actions = ["A", "B", "C"]
5 q = [2, 5, 3]
6 epsilon = 0.2
7 temperature = 1.0
8 # Epsilon-Greedy
9 if random.random() < epsilon:
10     eg_action = random.choice(actions)
11 else:
12     eg_action = actions[q.index(max(q))]
13 # Softmax
14 prob = [math.exp(x / temperature) for x in q]
15 total = sum(prob)
16 prob = [x / total for x in prob]
17 soft_action = random.choices(actions, weights=prob)[0]
18 print("Q-values:", q)
19 print("Epsilon-Greedy Action:", eg_action)
20 print("Softmax Action:", soft_action)
21 print("Softmax Probabilities:", prob)
```

```
Q-values: [2, 5, 3]
Epsilon-Greedy Action: B
Softmax Action: B
Softmax Probabilities: [0.042010066134066056, 0.8437947344813395, 0.11419519938459449]

=== Code Execution Successful ===
```

Comparison

Feature	ϵ -Greedy	Softmax
Exploration	Random action with probability ϵ	Probabilistic action selection
Exploitation	Selects highest Q-value	Favors higher Q-values
Control	ϵ controls exploration	Temperature controls exploration
Simplicity	Very simple	Slightly more complex
Learning Efficiency	Good for simple problems	Better when action values differ gradually
Main Advantage	Easy to implement	Uses Q-value differences
Main Limitation	Random exploration may select poor actions	Requires temperature tuning

Justification

The exploration–exploitation trade-off is important because an RL agent must **explore** unknown actions to discover better rewards while also **exploiting** actions that are already known to perform well.

In **ϵ -greedy**, the agent selects a random action with probability ϵ and the best-known action otherwise. It is simple and effective, but exploration is completely random.

In **Softmax**, actions are selected according to their Q-values. Actions with higher Q-values receive higher probabilities, so exploration is more informed. The temperature controls how strongly the agent favors the best action.

Therefore, ϵ -greedy is simpler and suitable for basic RL problems, while Softmax can provide more balanced and informed exploration when the differences between action values are important.

Constraints

- $0 \leq \epsilon \leq 1$
- Temperature must be greater than 0.
- Q-values must be numerical.
- At least two actions should be available.
- Random exploration should not be completely disabled when exploration is required.

3. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Solution:

Problem Statement

Identify the major challenges of applying **Reinforcement Learning (RL)** in **real-world robotics** and analyze how **safety, sample efficiency, and environment variability** affect the performance of a robotic agent.

Objectives

- Understand the challenges of RL in robotics.
- Analyze the importance of robot safety.
- Understand the effect of sample efficiency on training.
- Study the impact of changing environments.
- Identify methods to improve RL agent performance.

Eligibility

- Basic knowledge of Artificial Intelligence and Reinforcement Learning.
- Basic understanding of robotics.
- Basic Python programming knowledge.

Challenge Rules

- Identify at least three major challenges.
- Explain safety, sample efficiency, and environment variability.
- Analyze their impact on robot performance.
- Suggest suitable solutions for the identified challenges.
- Use a realistic robotics scenario.

Programming Languages

Python 3.x

Environment Setup

- Python 3.x
- Jupyter Notebook / Google Colab / GitHub

- No external libraries are required.

Input Format

The program takes simple values representing the robot's learning conditions.

Safety Risk: High

Training Samples: 100

Environment Changes: 3

Code

```
# RL Robotics Challenge Analysis

safety_risk = "High"
training_samples = 100
environment_changes = 3

print("RL Robotics Analysis")
print("-----")

print("Safety Risk:", safety_risk)

if safety_risk == "High":
    print("Effect: Agent may cause unsafe actions.")
else:
    print("Effect: Low safety risk.")

print("Training Samples:", training_samples)

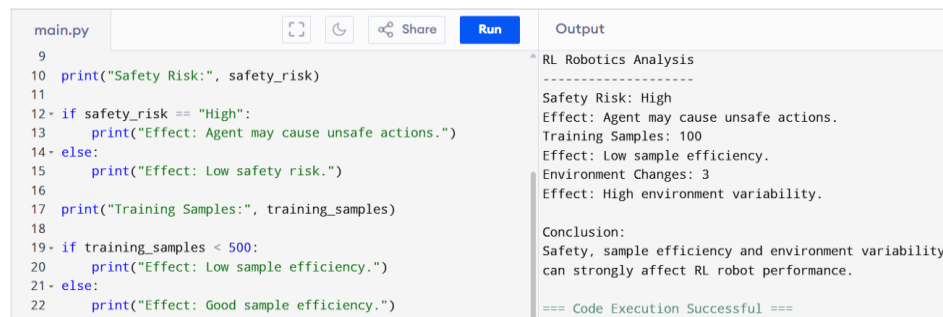
if training_samples < 500:
    print("Effect: Low sample efficiency.")
else:
    print("Effect: Good sample efficiency.")

print("Environment Changes:", environment_changes)

if environment_changes > 2:
    print("Effect: High environment variability.")
else:
    print("Effect: Low environment variability.")

print("\nConclusion:")
print("Safety, sample efficiency and environment variability")
print("can strongly affect RL robot performance.")
```


Output Format



```

main.py
9
10 print("Safety Risk:", safety_risk)
11
12 if safety_risk == "High":
13     print("Effect: Agent may cause unsafe actions.")
14 else:
15     print("Effect: Low safety risk.")
16
17 print("Training Samples:", training_samples)
18
19 if training_samples < 500:
20     print("Effect: Low sample efficiency.")
21 else:
22     print("Effect: Good sample efficiency.")
23

```

```

RL Robotics Analysis
-----
Safety Risk: High
Effect: Agent may cause unsafe actions.
Training Samples: 100
Effect: Low sample efficiency.
Environment Changes: 3
Effect: High environment variability.

Conclusion:
Safety, sample efficiency and environment variability
can strongly affect RL robot performance.

=== Code Execution Successful ===

```

Analysis

1. Safety

A robot learns by trial and error. In a real environment, incorrect actions can cause **collisions, equipment damage, or human injury**. Therefore, unrestricted exploration is dangerous.

Effect: Safety constraints can reduce the number of actions available to the agent and may slow learning.

Solution: Use safety constraints, simulation-based training, safe exploration, and emergency-stop mechanisms.

2. Sample Efficiency

RL generally requires many interactions to learn a good policy. Real robots are expensive and slow to operate compared with simulated environments.

Effect: Requiring too many real-world trials increases training time, cost, and mechanical wear.

Solution: Use simulation, transfer learning, offline data, and model-based RL to reduce real-world training requirements.

3. Environment Variability

Real-world environments can change because of **lighting, obstacles, objects, surfaces, people, and weather conditions**.

Effect: A policy learned in one environment may perform poorly when conditions change.

Solution: Use domain randomization, adaptive learning, robust policies, and continuous training.

Constraints

- Robot actions must remain within safe operating limits.
- Training data should be sufficient for reliable learning.
- The agent should handle environmental changes.
- Real-world testing should be performed carefully.
- The RL system should prioritize safety over reward maximization.

4. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Solution:

Problem Statement

Evaluate the effectiveness of **Model-Free Reinforcement Learning (RL)** and **Model-Based Reinforcement Learning** in dynamic environments. The task is to compare their strengths, limitations, learning process, and suitability for changing real-world conditions.

Objectives

- Understand model-free and model-based RL.
- Compare their learning approaches.
- Analyze their strengths and limitations.
- Study their performance in dynamic environments.
- Identify suitable applications for each approach.

Eligibility

- Basic knowledge of Artificial Intelligence and Reinforcement Learning.
- Basic understanding of states, actions, and rewards.
- Basic Python programming knowledge.

Challenge Rules

- Clearly differentiate model-free and model-based RL.
- Compare their advantages and limitations.
- Consider changing environmental conditions.
- Explain their effect on learning speed and performance.
- Provide a clear conclusion based on the comparison.

Programming Languages

Python 3.x

Environment Setup

- Python 3.x
- Jupyter Notebook / Google Colab / GitHub

- No external libraries are required.

Input Format

The input contains the reward values obtained by the two RL approaches.

Model-Free Rewards: 5, 7, 8, 10

Model-Based Rewards: 6, 8, 9, 12

Code

```
# Model-Free vs Model-Based RL

model_free = [5, 7, 8, 10]
model_based = [6, 8, 9, 12]

mf_avg = sum(model_free) / len(model_free)
mb_avg = sum(model_based) / len(model_based)

print("Model-Free Average Reward:", mf_avg)
print("Model-Based Average Reward:", mb_avg)

if mf_avg > mb_avg:
    print("Model-Free RL performs better.")
elif mb_avg > mf_avg:
    print("Model-Based RL performs better.")
else:
    print("Both methods perform equally.")
```

Output Format

```
1 # Model-Free vs Model-Based RL
2
3 model_free = [5, 7, 8, 10]
4 model_based = [6, 8, 9, 12]
5
6 mf_avg = sum(model_free) / len(model_free)
7 mb_avg = sum(model_based) / len(model_based)
8
9 print("Model-Free Average Reward:", mf_avg)
10 print("Model-Based Average Reward:", mb_avg)
11
12 if mf_avg > mb_avg:
13     print("Model-Free RL performs better.")
14 elif mb_avg > mf_avg:
15     print("Model-Based RL performs better.")
16 else:
17     print("Both methods perform equally.")
```

```
Model-Free Average Reward: 7.5
Model-Based Average Reward: 8.75
Model-Based RL performs better.
```

```
=== Code Execution Successful ===
```

Comparison

Feature	Model-Free RL	Model-Based RL
Environment Model	Not required	Required/learned
Learning	Learns directly from experience	Uses a model to predict outcomes
Sample Efficiency	Usually lower	Usually higher
Computation	Generally simpler during learning	More computation for planning
Adaptation	Can adapt directly through experience	May struggle if the model becomes inaccurate
Dynamic Environment	Good with sufficient new experience	Good when the environment model remains accurate
Main Strength	Simple and flexible	Planning and better sample efficiency
Main Limitation	Requires many interactions	Model can become inaccurate

Analysis

Model-Free RL learns the best action directly from experience without constructing an explicit model of the environment. It is flexible and can work well in complex environments, but it generally requires a large amount of training data.

Model-Based RL learns or uses a model of the environment to predict future states and rewards. This allows the agent to plan ahead and often improves **sample efficiency**. However, if the environment changes significantly, an inaccurate model can lead to poor decisions.

In **dynamic environments**, model-free RL can be more robust when the agent continuously receives new experience. Model-based RL can be more efficient when changes can be detected and the environment model is updated quickly.

Conclusion

Model-Free RL is simpler and flexible but may require more training experience. **Model-Based RL** provides better planning and sample efficiency but depends on the accuracy of its environment model. For highly dynamic environments, the best choice depends on how quickly the environment changes and how effectively the model can be updated.

5. Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

Solution:

Problem Statement

Differentiate between **on-policy** and **off-policy** learning methods in Reinforcement Learning (RL), and assess their advantages and limitations in practical applications. The objective is to understand how both methods learn from experience and determine where each approach is more suitable.

Objectives

- Understand on-policy and off-policy learning.
- Differentiate between the two approaches.
- Study their advantages and limitations.
- Compare their learning efficiency.
- Identify suitable practical applications.

Eligibility

- Basic knowledge of Artificial Intelligence and Reinforcement Learning.
- Basic understanding of states, actions, and rewards.
- Basic Python programming knowledge.

Challenge Rules

- Clearly differentiate on-policy and off-policy learning.
- Explain their advantages and limitations.
- Compare their learning process.
- Use suitable examples such as **SARSA** and **Q-Learning**.
- Provide a clear conclusion.

Programming Languages

Python 3.x

Environment Setup

- Python 3.x
- Jupyter Notebook / Google Colab / GitHub

- No external libraries are required.

Input Format

The program uses the current state, action, reward, next state, and next action.

State = 0

Action = 1

Reward = 10

Next State = 1

Next Action = 0

Code

On-Policy (SARSA) vs Off-Policy (Q-Learning)

```
Q = [[0, 0], [0, 0]]
```

```
state = 0
```

```
action = 1
```

```
reward = 10
```

```
next_state = 1
```

```
next_action = 0
```

```
alpha = 0.5
```

```
gamma = 0.9
```

```
# SARSA - On Policy
```

```
Q[state][action] += alpha * (
    reward + gamma * Q[next_state][next_action]
    - Q[state][action]
)
```

```
print("SARSA (On-Policy) Q-value:", Q[state][action])
```

```
# Q-Learning - Off Policy
```

```
Q[state][action] = 0
```

```
Q[state][action] += alpha * (
    reward + gamma * max(Q[next_state])
    - Q[state][action]
)
```

```
print("Q-Learning (Off-Policy) Q-value:", Q[state][action])
```

Output Format

```

6  action = 1
7  reward = 10
8  next_state = 1
9  next_action = 0
10
11  alpha = 0.5
12  gamma = 0.9
13
14  # SARSA - On Policy
15  Q[state][action] += alpha * (
16      reward + gamma * Q[next_state][next_action]
17      - Q[state][action]
18  )
19
20  print("SARSA (On-Policy) Q-value:", Q[state][action])
21
22  # Q-Learning - Off Policy
23  Q[state][action] = 0
24
25  Q[state][action] += alpha * (
26      reward + gamma * max(Q[next_state])
27      - Q[state][action]
28  )
29
30  print("Q-Learning (Off-Policy) Q-value:", Q[state][action])

```

* SARSA (On-Policy) Q-value: 5.0
 Q-Learning (Off-Policy) Q-value: 5.0
 === Code Execution Successful ===

Difference Between On-Policy and Off-Policy

Feature	On-Policy	Off-Policy
Learning	Learns the policy being followed	Learns a different target policy
Example	SARSA	Q-Learning
Exploration	Included in learning policy	Can learn optimal policy while exploring
Learning Data	Usually learns from current policy	Can learn from other policies/data
Sample Reuse	Less flexible	More flexible
Main Advantage	Safer and more consistent behavior	Efficient and can learn optimal policies
Main Limitation	Exploration can affect learning	Can be unstable in some situations

Advantages and Limitations

On-Policy – SARSA

Advantages:

- Learns from the actual behavior of the agent.
- Suitable when safe and conservative behavior is important.
- Considers the effect of exploration during learning.

Limitations:

- May learn more slowly.
- Exploration can affect the learned policy.
- Less flexible for reusing past experiences.

Off-Policy – Q-Learning

Advantages:

- Can learn the optimal policy while following an exploratory policy.
- Can reuse experiences effectively.
- Useful when exploration and optimal decision-making need to be separated.

Limitations:

- Can be less stable in complex environments.
- Learning may be sensitive to parameter selection.
- The learned policy may differ from the behavior used during training.

Practical Applications

On-policy learning is useful in applications such as **robot navigation and safety-sensitive control**, where learning from the agent's actual behavior is important.

Off-policy learning is useful in **recommendation systems, games, and large-scale decision-making**, where experience can be collected separately and reused for learning.

Conclusion

On-policy methods, such as SARSA, learn using the policy currently being followed and are suitable when safe and consistent behavior is important. **Off-policy methods**, such as Q-Learning, can learn an optimal policy independently of the behavior policy and provide greater flexibility and experience reuse. The appropriate method depends on the application's safety requirements, data availability, and complexity.

Code of Conduct:

*I **Yoga Madha A-192425058** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student :



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately